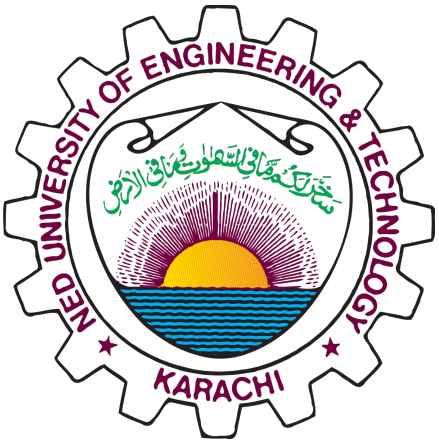


NED University
of Engineering and Technology

Team Reference Document

Team $(ttl)^3$

DP

0-1 Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can only be taken **once**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = W; j >= weights[i]; --j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Complete Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that any item can be taken **unlimited times**.

Time Complexity: $O(nW)$ **Space Complexity:** $O(W)$

```
int knapsack(int n, vector<int>& weights, vector<int>& values, int W) {
    vector<int> dp(W + 1, 0);
    for(int i = 0; i < n; i++) {
        for(int j = weights[i]; j <= W; ++j) {
            dp[j] = max(dp[j], values[i] + dp[W - j]);
        }
    }
    return dp[W];
}
```

Multiple Knapsack You are given n items with i_{th} item having weight w_i and value v_i. Given a knapsack with max Weight capacity W. Find the max value you can have.

Note that each item can be taken k_i **times**.

Time Complexity: $O(n \sum \log k_i)$ **Space Complexity:** $O(W)$

```
// Just break each item in powers of 2 and treat each division as single item and apply 01 knapsack
```

Edit Distance Given two strings s_1 and s_2, find the minimum number of operations (insert, delete, replace) required to transform s_1 into s_2.

Time Complexity: $O(mn)$ **Space Complexity:** $O(mn)$

```
int editDistance(string& s1, string& s2) {
    int m = s1.size(), n = s2.size();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1));
    for (int i = 0; i <= m; ++i) dp[i][0] = i;
    for (int j = 0; j <= n; ++j) dp[0][j] = j;
    for (int i = 1; i <= m; ++i) {
        for (int j = 1; j <= n; ++j) {
            dp[i][j] = min(dp[i - 1][j] + 1, dp[i][j - 1] + 1, dp[i - 1][j - 1] + (s1[i - 1] != s2[j - 1]));
        }
    }
    return dp[m][n];
}
```

Strings

Polynomial Hash

Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
const ll M = 991831889;
const ll C = uniform_int_distribution<ll>(0.1 * M, 0.9 * M)(rng);

struct HashString {
    int n;
    vector<ll> pows, sums;

    HashString(string s) : n(s.size()), pows(n + 1, 1), sums(n + 1) {
        for (int i = 1; i <= n; i++) {
            pows[i] = pows[i - 1] * C % M;
            sums[i] = (sums[i - 1] * C + s[i - 1]) % M;
        }
```

```
    }

    // Returns the hash of the substring [l, r)
    ll hash(int l, int r) {
        ll h = sums[r] - sums[l] * pows[r - l];
        return (h % M + M) % M;
    }
};
```

Trie

Time Complexity: $O(\sum |s_i|)$ **Space Complexity:** $O(26 * n)$

```
struct Vertex {
    int next[26];
    bool output = false;
    int link = -1;
    int p;
    char pch;
    int go[26];

    string text = "";
    int nearestOutput = -2;
    Vertex(int par = 0, char ch = '$'): p(par), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};
vector<Vertex> t(1);

int addString(string& s) {
    int v = 0;
    for(char ch: s) {
        int c = ch - 'a';
        if(t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
    t[v].text = s;
    return v;
}

int go(int v, char ch);

int getLink(int v) {
    if(t[v].link == -1) {
        if(v == 0 || t[v].p == 0) {
            t[v].link = 0;
        } else {
            t[v].link = go(getLink(t[v].p), t[v].pch);
        }
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if(t[v].go[c] == -1) {
        if(t[v].next[c] != -1) {
            t[v].go[c] = t[v].next[c];
        } else {
            t[v].go[c] = v == 0 ? 0: go(getLink(v), ch);
        }
    }
    return t[v].go[c];
}

int getNearestOutput(int v) {
    if(t[v].nearestOutput == -2) {
        if(v == 0) {
            t[v].nearestOutput = -1;
        } else {
            int u = getLink(v);
            if(u == 0) {
                t[v].nearestOutput = -1;
```

```
            } else if(t[u].output) {
                t[v].nearestOutput = u;
            } else {
                t[v].nearestOutput = getNearestOutput(u);
            }
        }
    }
    return t[v].nearestOutput;
}
```

Z-Algorithm
Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
vector<int> z_function(string& s) {
    int n = s.size();
    vector<int> z(n);
    int l = 0, r = 0;
    for(int i = 1; i < n; i++) {
        if(i < r) {
            z[i] = min(r - i, z[i - l]);
        }
        while(i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if(i + z[i] > r) {
            l = i;
            r = i + z[i];
        }
    }
    return z;
}
```

Manacher Algorithm
Time Complexity: $O(N)$ **Space Complexity:** $O(N)$

```
#include "bits/stdc++.h"
using namespace std;

vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2, 0);
    int l = 0, r = 1;
    for (int i = 1; i <= n; i++) {
        if (i <= r)
            p[i] = min(r - i, p[l + (r - i)]);
        while (s[i - p[i]] == s[i + p[i]])
            p[i]++;
        if (i + p[i] > r) {
            l = i - p[i];
            r = i + p[i];
        }
    }
    return vector<int>(p.begin() + 1, p.end() - 1);
}

vector<int> manacher(const string& s) {
    string t;
    t.reserve(2 * s.size() + 1);
    for (char c : s) {
        t += '#';
        t += c;
    }
    t += '#';
    return manacher_odd(t);
}

void solve()
{
    string s;
    cin >> s;
    string org = s;

    vi p = manacher(s);
    vi ans(org.size());
    vi right(org.size());
    f(i, 1, p.size() - 1) {
```

```
        int len = p[i] - 1;
        int pos = i / 2 + len / 2;
        if(i % 2 == 0) pos--;
        // Starting from pos with 'len' length palindrome
        right[pos] = max(right[pos], len);
    }

    int maxLen = 0;
    fr(i, org.size() - 1, -1) {
        maxLen = max(right[i], maxLen);
        ans[i] = maxLen;
        maxLen -= 2;
    }

    f(i, 0, org.size()) {
        cout << ans[i] << ' ';
    }

}
```

KMP KMP Automation

Time Complexity: $O(N)\&O(N.M)$ **Space Complexity:** $O(N)\&O(N.M)$

```
#include "bits/stdc++.h"

using namespace std;

void kmpAutomation(int n, string& p) {
    int m = p.size();

    // Step 1: Compute KMP pi array
    vector<int> pi(m);
    for (int i = 1; i < m; i++) {
        int j = pi[i - 1];
        while (j > 0 && p[i] != p[j])
            j = pi[j - 1];
        if (p[i] == p[j])
            j++;
        pi[i] = j;
    }

    // Step 2: Build String Automaton
    vector<vector<int>> aut(m, vector<int>(26));
    for (int j = 0; j < m; j++) {
        for (int c = 0; c < 26; c++) {
            if (j > 0 && 'A' + c != p[j])
                aut[j][c] = aut[pi[j - 1]][c];
            else
                aut[j][c] = j + ('A' + c == p[j]);
        }
    }

    // Step 3: DP Table
    // dp[i][j] = ways to build a string of length i ending in automaton state j
    vector<vector<long long>> dp(n + 1, vector<long long>(m + 1, 0));
    dp[0][0] = 1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j <= m; j++) {
            if (dp[i][j] == 0) continue;

            if (j == m) {
                // Absorbing state: once matched, any character keeps it matched
                dp[i + 1][m] = (dp[i + 1][m] + dp[i][m] * 26LL) % MOD;
            } else {
                // Transition to next state for all 26 characters
                for (int c = 0; c < 26; c++) {
                    int nxt = aut[j][c];
                    dp[i + 1][nxt] = (dp[i + 1][nxt] + dp[i][j]) % MOD;
                }
            }
        }
    }

    cout << dp[n][m] << "\n";
}
```